

# Technical document

As-built local prototype | Architecture, controls and verification

## System overview

The web app is a local FastAPI service with a plain JavaScript interface and SQLite persistence. A server-side adapter calls the configured Ollama-compatible gateway at POST /api/chat. Live mode uses model-selected actions; offline mode uses labeled deterministic selections. No cloud hosting, managed knowledge base or HR integration is asserted.

| Component         | Implemented responsibility                                                                                      |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| Browser interface | Learning path, source-grounded chat, trainer studio, evidence dashboard and rehearsal controls.                 |
| FastAPI routes    | Authenticate requests, enforce roles and validate typed request bodies.                                         |
| Agent layer       | Bounded chat tool loop and eligible-activity selection; malformed JSON receives limited repair.                 |
| Workflow layer    | Source approval, course publication, activity eligibility, scoring, revision activation and evidence carryover. |
| SQLite            | Users, sessions, sources, courses, attempts, chats, learning evidence, reviews and revision lineage.            |
| Model gateway     | Remote inference using server-side configuration; credentials are not returned to the browser.                  |

## Two execution paths

Chat: authenticated question -> model chooses an allowed read tool -> tool result -> model answer -> citation/source checks -> saved response. Learning: saved session -> eligible choices -> model selection -> transactional revalidation -> issued activity -> learner submission -> backend outcome.

## Trust boundary

The model does not grade answers, approve sources, publish courses or activate revisions. The trainer uses separate authenticated routes for those approvals. Source text is reference data, not executable instructions. The browser displays backend readiness rather than inferring it from a quiz percentage.

# Agent contract and learning

Constrained tools and server-authoritative evidence

| Allowed read tool        | Purpose                                                                    |
|--------------------------|----------------------------------------------------------------------------|
| get_learning_state       | Read the signed-in learner's readiness sessions and eligible actions.      |
| select_approved_activity | Recommend a currently eligible activity; chat does not issue or answer it. |
| get_my_progress          | Read permitted course and attempt information.                             |
| retrieve_sources         | Search approved passages using lexical overlap.                            |
| get_course_outline       | Read an approved course outline without answer keys.                       |
| recommend_lesson         | Read an approved lesson and its source reference.                          |

## Bounded execution and failure behavior

Pydantic schemas reject unknown tools, unexpected fields and invalid argument types. Chat permits at most five model turns, including final output and any format repair. Learning-path selection permits at most two model calls when JSON repair is needed. Responses use a 512-token output budget and a 60-second transport timeout. Invalid output is not executed; errors are explicit rather than silently switching to offline mode.

## Readiness rules

Diagnostic answers identify gaps but do not demonstrate an objective. Reviewing a lesson also awards no readiness. Fresh reassessment evidence must demonstrate at least 80% of objectives and every critical objective. In the four-objective pilot this requires all four. Two available fresh cases per objective bound automatic practice; exhaustion creates a trainer-review item. Trainer guidance does not change scores.

## Persistence and revalidation

Each user/course has one learning session and at most one pending activity. The backend checks approval and eligibility when issuing and checks the issued activity when scoring. An identical resubmission is idempotent; a changed answer to an already submitted activity is rejected. Answer keys and future case banks are excluded from learner state. Decision records preserve observable evidence at selection and join the later submitted outcome.

# Version changes and isolation

Human approval, preserved history and separate rehearsals

## Procedure revision workflow

A trainer creates an unapproved source/course revision. Deterministic comparison inspects objective content, supporting source sections and completion rules. Changed or new objectives must be refreshed. The trainer reviews the source, lessons and cases and explicitly approves the mapping before activation.

Activation checks stored fingerprints and rejects stale or superseded bases. It publishes the new version, retires the prior source, creates successor sessions, links demonstrated unchanged objectives as carryover, and assigns changed objectives for refresh. Original attempts are not rewritten. Reused case prompts are excluded by exact normalized text matching; semantic paraphrase detection is not implemented.

| Record group                               | Purpose                                                              |
|--------------------------------------------|----------------------------------------------------------------------|
| learning_sessions / learning_activities    | User/course state and issued/submitted evidence.                     |
| learning_decisions / learning_reviews      | Selection evidence snapshots and saved trainer requests/resolutions. |
| procedure_revisions / procedure_successors | Draft/activation metadata, fingerprints and version lineage.         |
| learning_carryovers / learning_refreshes   | Accepted prior evidence and assigned procedure refresh requirements. |

## Rehearsal isolation

A trainer can create a new fictional rehearsal in live or offline mode. Each rehearsal receives a separate SQLite file and original seed policy. Request-local ContextVars select the database and model mode; they do not change the process-wide database for other requests. Cookies select the rehearsal and preserve the main login for return. Browser tabs share that selection. Exiting retains the rehearsal file and restores the main workspace session.

## Access and operational limits

Learners see their own progress; trainers can inspect team evidence and approve content. Sessions use HttpOnly, SameSite cookies, with Secure enabled for HTTPS. POST origin checks and request-size bounds are implemented. Shared demonstration credentials and local SQLite are suitable for the prototype only. Managed identity, tenant authorization, rate limits, backups and deployment hardening remain required before public use.

# Reproduction and verification

Evidence recorded through 27 September 2026

## Local setup

Use Python 3.12 or newer. In employee-training-assistant, create .venv with `python -m venv .venv`, install requirements.txt, then run `./start.ps1 -Port 8013`. Open `http://127.0.0.1:8013`. The shared fictional accounts are learner, alex and trainer; local demo password: LearnDemo2026!.

Configure AGENT\_MODE as demo or gateway. Live mode requires LLM\_GATEWAY\_URL, LLM\_GATEWAY\_API\_KEY and LLM\_MODEL in the ignored local .env. TRAINING\_DB optionally sets the database path. Never include .env, session cookies, learner databases or gateway logs in a shared submission.

## Verification commands

Python suite: `.venv/Scripts/python.exe -m pytest -q`

Dashboard checks: `node --test tests/dashboard.test.cjs`

Opt-in live check: `.venv/Scripts/python.exe -m src.validate_live`

Health endpoint: `GET /health`

| Evidence             | Observed result / limit                                                                                                                                                            |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Automated tests      | 71 Python tests and 2 dashboard aggregation tests passed in the latest recorded run. Python run reported two dependency deprecation warnings.                                      |
| Gateway smoke checks | Three consecutive runs passed activity selection, cited policy response and unsupported-question referral after bounded format recovery. Earlier output/timeout failures occurred. |
| Browser rehearsal    | Original policy reached ready; revision carried three objectives, refreshed one, then reached ready on version two.                                                                |
| Timing               | Resumed segment approximately 2m 47s; excludes prior diagnostic work and interruption. Not a continuous full-demo benchmark.                                                       |

## Known limitations and deployment evidence

Lexical retrieval can miss paraphrases. Citation identifiers prove neither semantic support nor learning effectiveness. No short-answer grading, semantic change analysis or retention study is implemented. The demonstrated deployment is localhost, not a judge-accessible public URL. A repository URL, actual demo video and deployment evidence package still need submission-specific completion.

Implementation references: `src/main.py`; `src/agents/{protocol,orchestrator,learning_coach,worker}.py`; `src/workflows/{readiness,change_impact,rehearsal}.py`; `src/core/{state,config}.py`; `tests/`. Reproduction narrative: README.md, DEMO.md, REHEARSAL.md. Requirements source: organizer screenshot supplied by the user.